

Loyola Marymount University

CMSI 3300 Artificial Intelligence

Fall 2024

Professor Lanyu Shang

Announcements

- Classwork 6
  - Due **Tuesday** 11/26 @5pm
- Homework 4
  - Due Friday 12/6 @5pm
  - Check Brightspace for the updated GitHub Classroom link

This Lecture

- Logistic Regression
  - Optimization
  - Learning

Last Lecture

	Linear Perceptron	Logistic Regression
Input	$\boldsymbol{x} =< x_1, x_2, \cdots, x_n >$	$\boldsymbol{x} =< x_1, x_2, \cdots, x_n >$
Weight Vector	$\boldsymbol{w} =< w_1, w_2, \cdots, w_n >$	$\boldsymbol{w} =< w_1, w_2, \cdots, w_n >$
Activation	$\alpha = \boldsymbol{w} \cdot \boldsymbol{x} = \sum_i w_i \cdot x_i$	Binary: $\alpha = \sigma(z) = \frac{1}{1+e^{-z}}$, $z = \boldsymbol{w} \cdot \boldsymbol{x}$ Multi-Class: $\frac{e^{z_y}}{\sum_{y'} e^{z_{y'}}}$, $z_y = \boldsymbol{w_y} \cdot \boldsymbol{x}$
Decision Rule	Binary: $\hat{y} = (\alpha_{\boldsymbol{w}}(\boldsymbol{x}) > 0)?\, 0 : 1$ Multi-Class: $\hat{y} = \text{argmax}_y \alpha_y = \text{argmax}\, \boldsymbol{x} \cdot \boldsymbol{w_y}$	Binary: $\widehat{y^{(i)}} = \left(P\left(y^{(i)} = +1 \mid \boldsymbol{x}^{(i)}; \boldsymbol{w}\right) > 0.5\right)? +1 : -1$ Multi-Class: $\widehat{y^{(i)}} = \text{argmax}_{y^{(i)}} P\left(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w_y}\right)$
Optimization	For each misclassification: $\boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}$	?

Logistic Regression

- Objective
  - $\boldsymbol{w}^* = \text{argmax}_{\boldsymbol{w}} \prod_i P(y^{(i)}|\boldsymbol{x}^{(i)}; \boldsymbol{w})$
    $= \text{argmax}_{\boldsymbol{w}} \sum_i \log P(y^{(i)}|\boldsymbol{x}^{(i)}; \boldsymbol{w})$
  - $ll(\boldsymbol{w}) = \log P(y^{(i)}|\boldsymbol{x}^{(i)}; \boldsymbol{w})$
- Optimization
  - Maximizing $ll(\boldsymbol{w})$ as hill climbing
  - Equivalent to minimizing $-ll(\boldsymbol{w})$, descending downhill
  - Empirically, most optimization algorithms are designed to **minimize** functions.

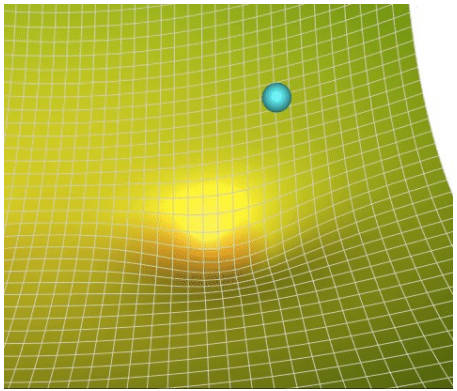
Optimization

- Partial Derivative
  - Given an objective $ll(\boldsymbol{w})$ where $\boldsymbol{w} =< \mathrm{w}_1, \mathrm{w}_2 >$, the partial derivative of $ll(\boldsymbol{w})$ is $\frac{\partial ll(\boldsymbol{w})}{\partial w_i}$
    - Treat other variables w_j s. t. $j \neq i$ as **constant**
    - Tell the direction of slope for w_i at a time
  - Example: $g(w_1, w_2) = 5w_1^2 + 4w_2$
    - $\frac{\partial g(w_1,w_2)}{\partial w_1} = 10w_1$ and $\frac{\partial g(w_1,w_2)}{\partial w_2} = 4$

Optimization

- Gradient
  - The combination of individual partial derivatives of a vector
  - Formally, let $g = -ll(\boldsymbol{w})$, ∇g is defined as:

$$\nabla g = \begin{bmatrix} \frac{\partial g}{\partial w_1} \\ \frac{\partial g}{\partial w_2} \\ \cdots \\ \frac{\partial g}{\partial w_n} \end{bmatrix}$$



Visualization of Gradient Descent

Image: https://miro.medium.com/v2/resize:fit:956/format:webp/1*hUd744hDEEGx0-ypWGhrkw.gif

Learning

- From training data: pairs of $(\boldsymbol{x}^{(i)}, y^{(i)})$
- Loss function
  - $Loss(\text{predicted}, \text{true}) = Loss(\hat{y}^{(i)}, y^{(i)})$
  - It provides a numerical metric of how wrong the model is.
- Cross Entropy (CE) Loss
    - $Loss_{CE}(\hat{y}^{(i)}, y^{(i)}) = -\log P(\hat{y}^{(i)}|\boldsymbol{x}^{(i)}; \boldsymbol{w})$
    - Commonly used for classification

Example

- 3 classes {0,1,2}, compute $Loss_{CE}(\hat{y}^{(i)}, y^{(i)})$ for each sample i.

Sample i	$P(y^{(i)}\|\boldsymbol{x}^{(i)};\boldsymbol{w})$	$y^{(i)}$ (true class)
i=1	⟨0.3,0.5,0.2⟩	0
i=2	⟨0.2,0.7,0.1⟩	1
i=3	⟨0.0,1.0,0.0⟩	1

$$Loss_{CE}(\hat{y}^{(1)}, y^{(1)}) = -\log(0.3) \approx 0.52$$

$$Loss_{CE}(\hat{y}^{(2)}, y^{(2)}) = -\log(0.7) \approx 0.15$$

$$Loss_{CE}(\hat{y}^{(3)}, y^{(3)}) = -\log(1) \approx 0$$

Gradient Descent

- Weight Update
  - Goal: maximize $ll(\boldsymbol{w})$ => **minimize** loss function
  - The derivative of a **loss function** w.r.t. **weights** indicates the direction to *nudge* those weights to be better!
- Derivative of $Loss_{CE}$ (L_{CE}) w.r.t. a class k and a sample i

$$\frac{\partial L_{CE}}{\partial w_{k,i}} = -\left[1\left(k = y^{(i)}\right) - P\left(\hat{y}^{(i)} = k \middle| \boldsymbol{x}^{(i)}; \boldsymbol{w_k}\right)\right] \cdot \boldsymbol{x}^{(i)}$$

Direction to move $w_{k,i}$ to

Indicator function =1 if $k = y^{(i)}$, =0 otherwise

Tweak proportional to the contribution of each feature

Gradient Descent

- Logistic Regression Weight Update Rule:
  - For a single sample $(x^{(i)}, y^{(i)})$, weights belonging to each class w_y can be nudged in the "more correct" direction
  - $w_{k,i} = w_{k,i} - \eta \cdot \frac{\partial L_{CE}}{\partial w_{k,i}}$

Current weight

Learning Rate

Direction

  - Demo: https://uclaacm.github.io/gradient-descent-visualiser/

Stochastic Gradient Descent (SGD)

- Stochastic: “random” training samples
  - Why not all training samples?
    - Computational infeasible
- Gradient: “best” direction to tweak all weights
- Descent: take step that descend the loss function

Stochastic Gradient Descent (SGD)

- Steps

```
initialize w_y = <0, 0, 0, ...>

# avoid over training
while validation set accuracy still increasing:
  for each sample (X^(i), y^(i)) in training set (in random order):
    # compute the current prediction
    compute P(Y|x^(i); w)
    # gradient
    compute gradient (derivatives) of Loss_CE for each w_{k,i}
    # update
    update weight of each class: w_{k,i} = w_{k,i} - learning_rate * gradient
```

Example

- 3 classes {0,1,2}, 2 features, 2 weights per class
- Current weight
  - $\boldsymbol{w_0} =< -1, 1 >, \boldsymbol{w_1} =< 1, -2 >, \boldsymbol{w_2} =< -2, 2 >$
- Find updated weights from sample:
  - $\boldsymbol{x}^{(i)} =< 2,1 >, y^{(i)} = 1, \eta = 0.1$

Example

- Step 1: Compute $P\left(\hat{y}^{(i)} \middle| \boldsymbol{x}^{(i)}; \boldsymbol{w_k}\right) \forall\ \hat{y} \in Y$
  - Step 1.1: Compute $z = \boldsymbol{w} \cdot \boldsymbol{x}$
    - $z_0 =< -1, 1 >\cdot< 2,1 > = -1$
    - $z_1 =< 1, -2 >\cdot< 2,1 > = 0$
    - $z_2 =< -2, 2 >\cdot< 2,1 > = -2$
  - Step 1.2: Softmax
    - $P\left(\hat{y}^{(i)} = k \middle| \boldsymbol{x}^{(i)}; \boldsymbol{w_k}\right)$

$$= \frac{e^{z_k}}{\sum_{y'} e^{z_{y'}}} =< \frac{0.36}{0.36 + 1.00 + 0.14}, \frac{1.00}{0.36 + 1.00 + 0.14}, \frac{0.14}{0.36 + 1.00 + 0.14} > \approx< 0.24, 0.67, 0.09 >$$

Example

- Step 2: Compute gradient of the loss function

$$\frac{\partial L_{CE}}{\partial w_{k,i}} = -\left[1\left(k = y^{(i)}\right) - P\left(\hat{y}^{(i)} = k \middle| \boldsymbol{x}^{(i)}; \boldsymbol{w_k}\right)\right] \cdot \boldsymbol{x}^{(i)}$$

k=0: $\frac{\partial L_{CE}}{\partial w_{k,i}} = -[0 - 0.24] \cdot < 2,1 > = < 0.48,0.24 >$

k=1: $\frac{\partial L_{CE}}{\partial w_{k,i}} = -[1 - 0.67] \cdot < 2,1 > = < -0.66, -0.33 >$

k=2: $\frac{\partial L_{CE}}{\partial w_{k,i}} = -[0 - 0.09] \cdot < 2,1 > = < 0.18, 0.09 >$

Example

- Step 3: Update weights of each class

$$w_{k,i} = w_{k,i} - \eta \cdot \frac{\partial L_{CE}}{\partial w_{k,i}}$$

$$w_0 = < -1,1 > -0.1 * < 0.48,0.24 > = < -1.048,0.976 >$$

$$w_1 = < 1,-2 > -0.1 * < -0.66,-0.33 > = < 1.066,-1.976 >$$

$$w_2 = < -2,2 > -0.1 * < 0.18,0.09 > = < -2.018,1.991 >$$